

Datos del estudiante

Nombre y apellidos Grace Karol Huertas

Fecha de entrega 16/03/2026

Índice

1. Introducción	[Pág 2]
2. Reparto de Roles y Coordinación	[Pág 2]
3. Análisis del Sistema	[Pág 3]
• 3.1. Identificación de Actores	
• 3.2. Casos de Uso Principales	
4. Diseño de Diagramas UML (Justificación Técnica)	[Pág 4 - 8]
• 4.1. Diagrama de Casos de Uso	
• 4.2. Diagrama de Clases (Modelo de dominio)	
• 4.3. Diagrama de Secuencia (Agendar cita)	
• 4.4. Diagrama de Actividad (Flujo de agendar cita)	
• 4.5. Diagrama de Estados (Ciclo de vida de la cita)	
5. Implementación en Herramienta CASE	[Pág 9]
• 5.1. Proceso de Modelado en Visual Paradigm	
• 5.2. Generación de Código Fuente Java	
6. Reflexiones y Conclusiones	[Pág 10]
• 6.1. Ventajas e Inconvenientes de la Herramienta	
• 6.2. Valoración del Trabajo en Equipo	

1. Introducción

Este documento detalla el modelado completo de un sistema de gestión de citas médicas para una clínica. El objetivo es transformar una especificación funcional en un diseño técnico robusto utilizando un modelo de lenguaje unificado (UML).

Este diseño ha sido desarrollado priorizando la trazabilidad y la coherencia técnica, no tengo aún mucha idea de programación orientada a objetos, pero he priorizado la coherencia en mis diagramas a lo largo de todo el trabajo.

Justifico como cada interacción de los usuarios (casos de uso), se traduce en una estructura de datos sólida (diagrama de clases), y en comportamientos dinámicos (diagramas de secuencia, actividad y estados). El resultado final es una representación visual, además de una arquitectura preparada para la generación de código en java, he intentado garantizar la escalabilidad, eficiencia, y que se cumplan las reglas de negocio en un entorno clínico según lo propuesto por la actividad.

2. Reparto de roles y coordinación

Este proyecto se presenta de forma individual (además de la entrega grupal) debido a mi inconformidad con la coherencia técnica en el trabajo del grupo. Pese a que sugerí mejoras y cambios necesarios para cumplir con los objetivos de la asignatura, no percibí interés en llevarlas a cabo.

Al no estar conforme con el enfoque del grupo, he asumido todos los roles para asegurar un resultado profesional:

- **Diseño y Análisis:** Estructura de actores, casos de uso y clases.
- **Comportamiento:** Modelado de secuencia, actividad y estados.
- **Herramienta CASE:** Gestión en Visual Paradigm y generación de código Java.
- **Documentación:** Justificación técnica y redacción del informe.

3. Análisis del sistema

3.1 Identificación de actores

Paciente: Usuario final que gestiona sus citas e informes.

Médico: Profesional que realiza la atención y genera diagnósticos.

Administrador: Encargado de gestionar el personal y la clínica.

Sistema SMS (Externo): Servicio que envía las notificaciones automáticas.

3.2 Casos de uso principales

Gestión de citas: Procesos de agendar, consultar y cancelar.

Gestión clínica: Atención al paciente, generación de informes y consulta de historial.

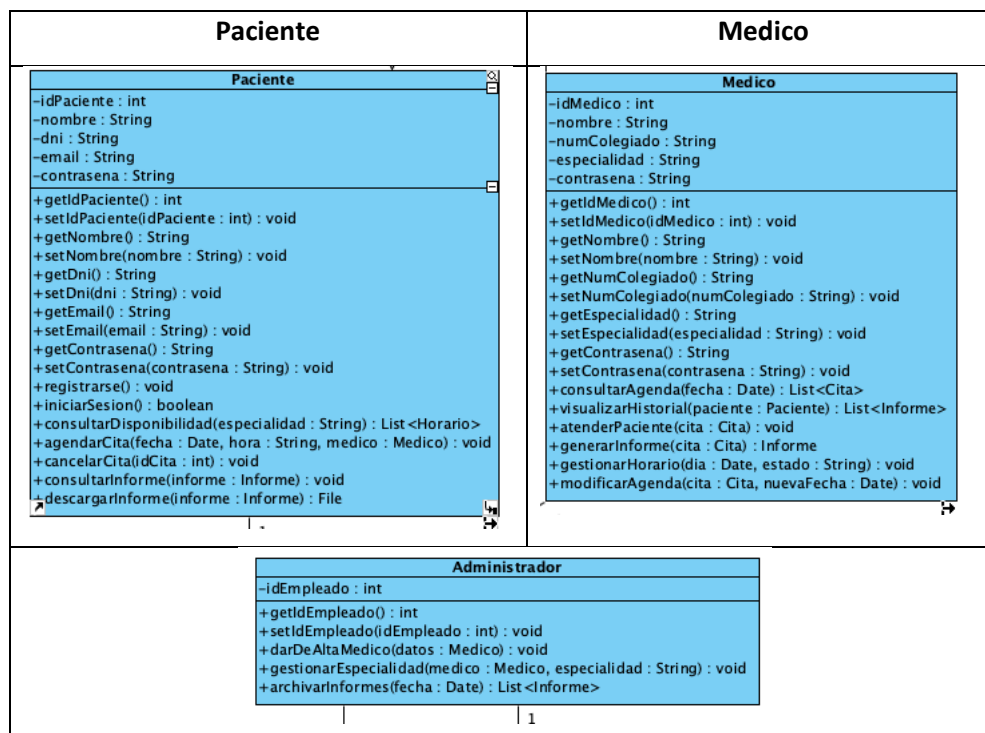
Gestión administrativa: Alta de médicos y control de especialidades.

3.3 Elementos clave: Las entidades

Clases: Paciente, Medico, Administrador, Cita, Informe.

Atributos: Datos de identidad, especialidades, fechas y estados de cita.

Métodos: agendarCita(), cancelarCita(), generarInforme().. etc.



4. Diseño de diagramas UML

4.1 Diagrama de casos de uso

Este diagrama define el alcance funcional de la aplicación. Se ha aplicado una relación de **Generalización** donde el Administrador hereda del Paciente, permitiéndole gestionar sus propias citas sin duplicar datos. (Y porque el administrador también podría ser un paciente). Se utilizan relaciones de **Inclusión** (<<include>>) para garantizar que toda cita genere una notificación SMS y que toda atención médica resulte en un informe obligatorio. Por otro lado, las extensiones (<<extend>>) representan funciones opcionales que solo ocurren si el usuario lo decide, como visualizar el historial previo durante una cita o descargar el archivo del informe una vez consultado; esto permite que el sistema sea flexible y no fuerce pasos innecesarios.

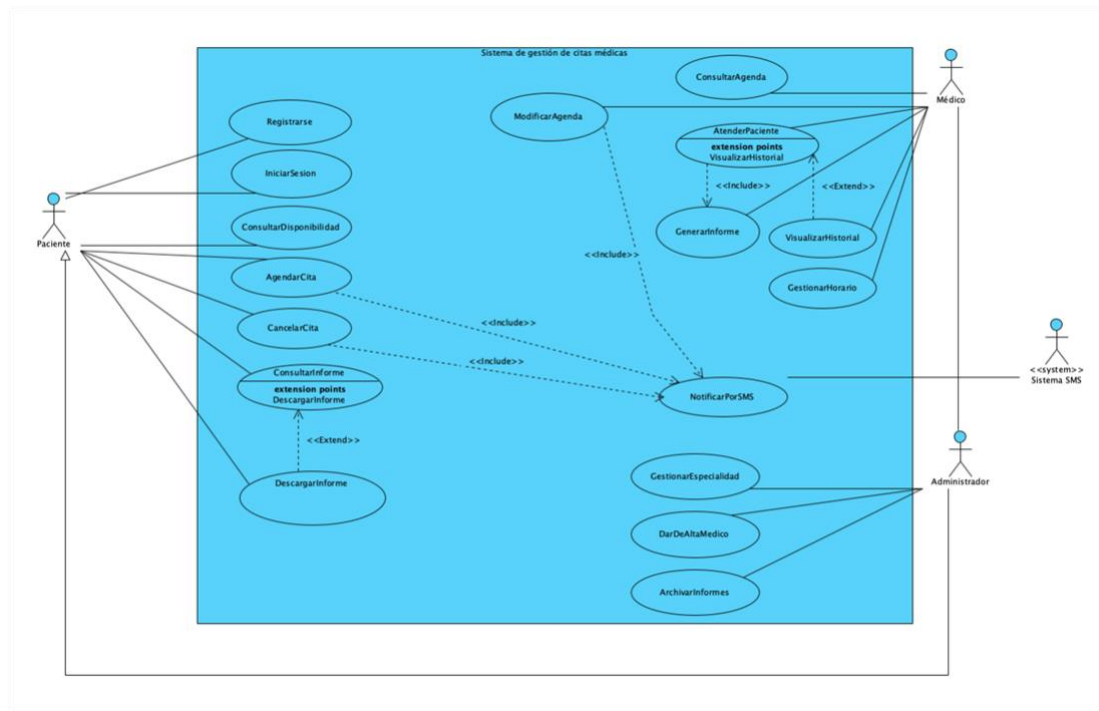


Ilustración 1: Diagrama de casos de uso

4.2 Diagrama de clases (modelo dominio)

Este diagrama estructura la persistencia y el comportamiento del sistema bajo principios de la programación orientada a objetos, (lo cual aún estoy aprendiendo). Destaca las relaciones que planteo más abajo, asegurando, por ejemplo, que el informe solo exista si la consulta se realiza. Se aplica un encapsulamiento estricto (atributos privados) para garantizar la integridad de los datos.

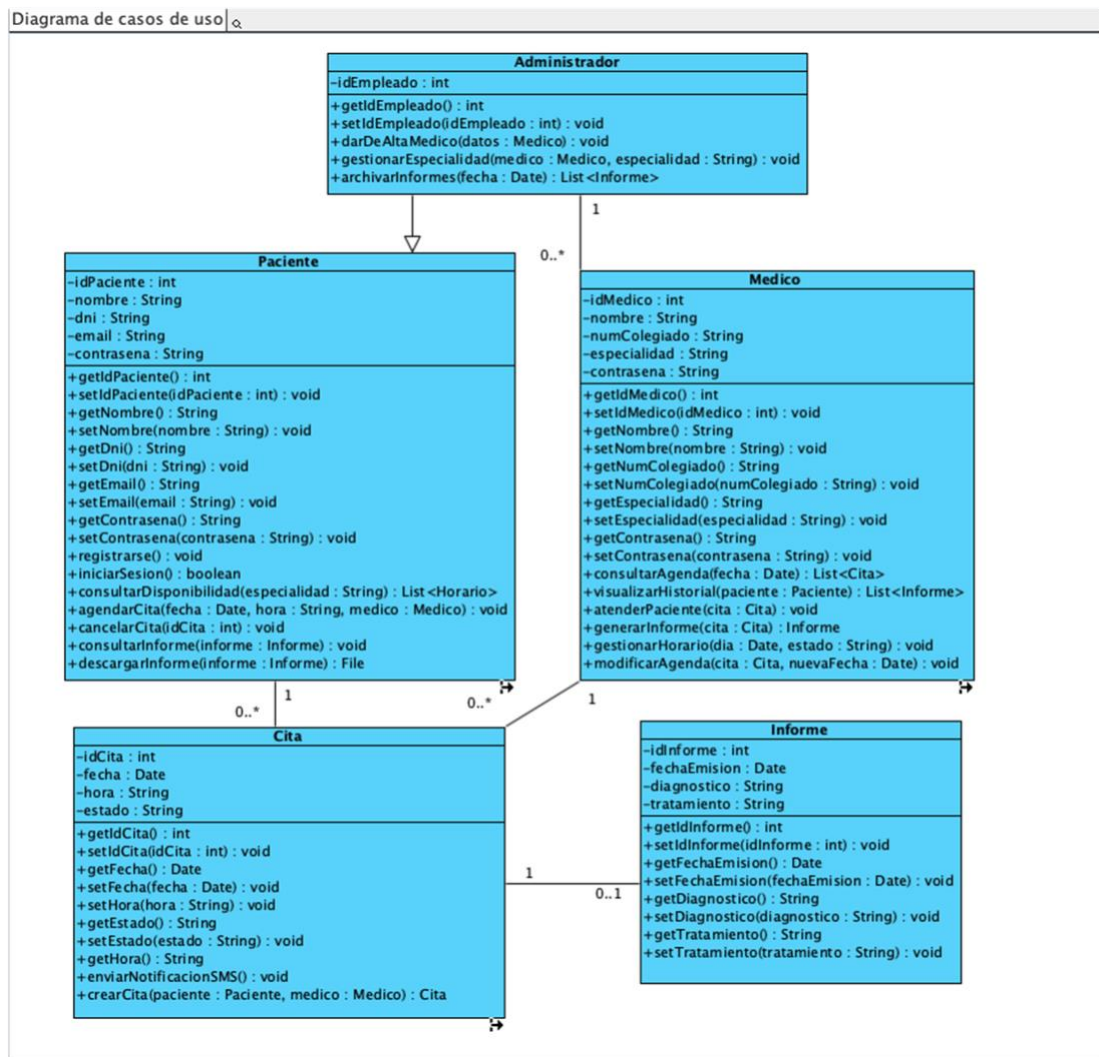


Ilustración 2: Diagrama de clases

Diccionario de clases (entidades del dominio)

Clase	Responsabilidad Principal	Relacion de herencia
Paciente	Representa al usuario que consume servicios. Gestiona su perfil, solicita y cancela citas, y accede a sus resultados médicos.	Ninguna (clase base)
Medico	Profesional sanitario que gestiona su agenda diaria, atiende consultas y emite diagnósticos (informes).	Ninguna (clase base)
Administrador	Encargado de la gestión operativa (altas de médicos y archivo). Actúa como supervisor del sistema.	Hereda de Paciente, pues un administrador también puede ser un paciente.
Cita	Actúa como el núcleo de interacción. Vincula a un paciente con un médico en un tiempo determinado.	Ninguna
Informe	Documento clínico que contiene el diagnóstico y tratamiento derivado de una cita celebrada.	Ninguna

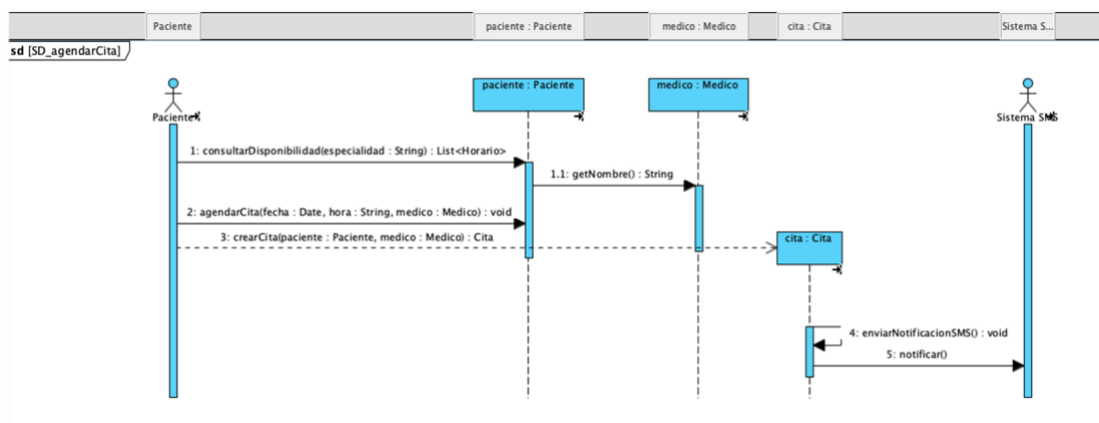
- **Nota de Diseño:** Se ha implementado una relación de **Generalización** entre Administrador y Paciente. Esta decisión evita la redundancia de datos, permitiendo que el Administrador utilice toda la lógica de gestión de citas y consulta de informes para su uso personal sin necesidad de duplicar atributos como DNI, Email o contraseña.

Tabla de relaciones y multiplicidades

Relación	Multiplicidad	Justificación
Paciente - Cita	1:0..*	1 Cita debe pertenecer obligatoriamente a 1 Paciente. El Paciente puede tener muchas citas históricas o ninguna si es nuevo.
Medico – Cita	1: 0..*	Cada Cita es asignada a 1 único Médico responsable. El Médico tiene una agenda con múltiples citas asignadas.
Cita – Informe	1:0..1	Una Cita genera como máximo 1 Informe (0 si la cita aún no ocurre o se cancela). El Informe no existe sin 1 Cita previa.
Administrador – Medico	1:0..*	1 Administrador tiene la potestad de gestionar y dar de alta a múltiples Médicos en la plantilla de la clínica.

4.3 Diagrama de secuencia (agendar cita)

Este diagrama detalla el proceso técnico de reserva. El flujo muestra cómo el objeto **Paciente** valida la disponibilidad del **Médico** e instancia una nueva **Cita**. El proceso culmina con la activación automática de una señal hacia el **Sistema SMS** para notificar al usuario.

*Ilustración 3: Diagrama de secuencia*

4.4 Diagrama de actividad

(flujo de agendar la cita)

Este diagrama describe el flujo lógico del algoritmo de agendar la cita. Incluye un **nodo de decisión** crítico: si el horario no está disponible, el sistema obliga a reajustar la búsqueda. Esto garantiza que no existan colisiones de horarios en la agenda de la clínica.

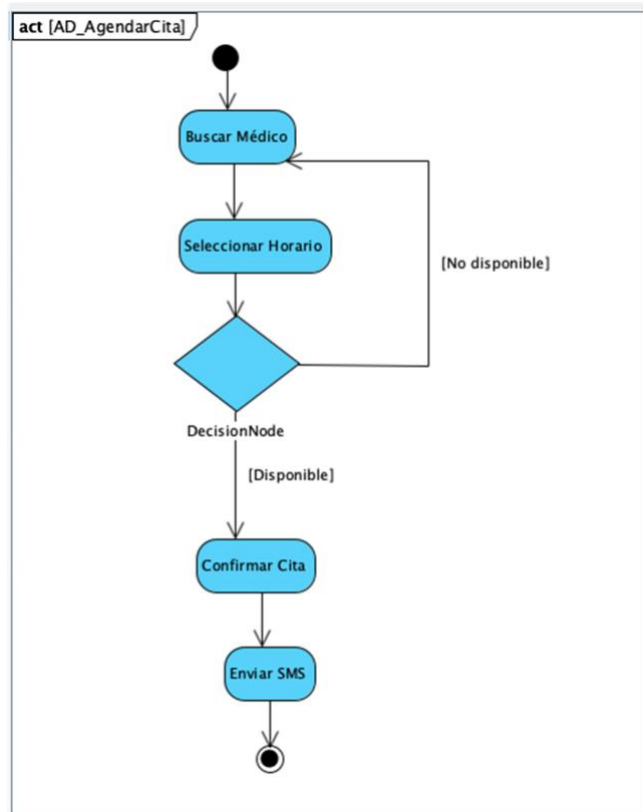
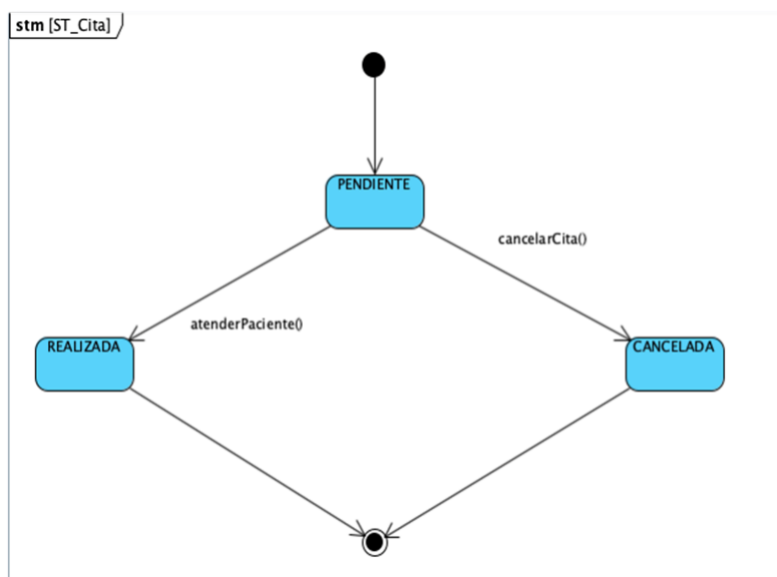


Ilustración 4: Diagrama de actividad



4.5 Diagrama de estados

(ciclo de vida de la cita)

Este diagrama representa los cambios de condición del objeto Cita. El ciclo inicia en **PENDIENTE** y transita a **REALIZADA** o **CANCELADA** mediante los métodos `atenderPaciente()` o `cancelarCita()`.

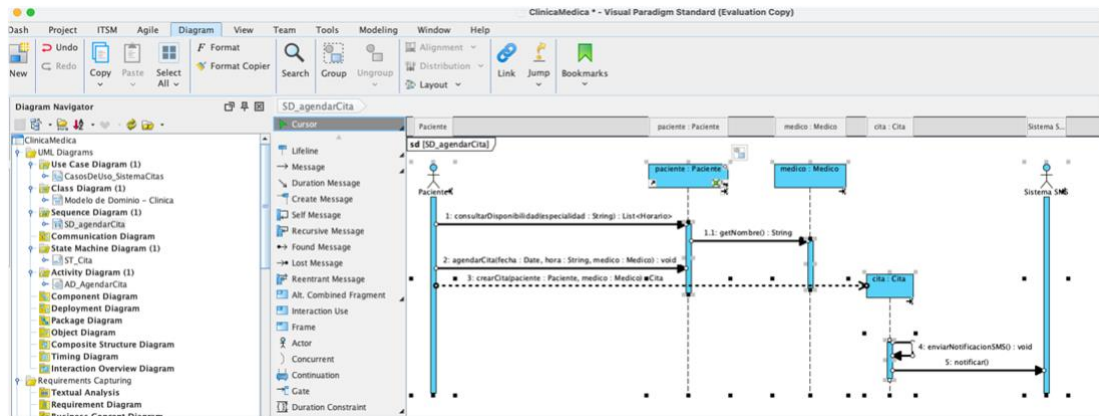
cancelarCita(). Estos estados finales aseguran la trazabilidad completa del historial médico.

Ilustración 5: Diagrama de estados

5. Implementación en herramienta CASE

5.1 Proceso de modelado en Visual Paradigm.

Tal como se había propuesto en el trabajo grupal, decidí implementar la misma herramienta CASE. Visual Paradigm. Me pareció bastante completa, intuitiva y profesional.



Ventajas clave del uso de la herramienta:

- **Sincronización:** Cualquier cambio en el diagrama de clases se refleja automáticamente en la estructura de los demás diagramas de comportamiento.
- **Validación UML:** Una de las cosas que más me gustaron de la herramienta es que impide errores de sintaxis comunes, como conectar elementos incompatibles en el diagrama de actividad.

5.2 Generación de código

A partir del **Diagrama de Clases**, se ha utilizado la función *Update Code* para generar el esqueleto de la aplicación. Este proceso garantiza que los atributos, métodos y relaciones de herencia definidos en UML se implementen con precisión en el código. (adjuntas todas las clases)

```

J Paciente.java > Paciente
1  public class Paciente {
2
3      private int idPaciente;
4      private String nombre;
5      private String dni;
6      private String email;
7      private String contraseña;
8
9      public int getIdPaciente() {
10         return this.idPaciente;
11     }
12
13     /**
14      *
15      * @param idPaciente
16      */
17     public void setIdPaciente(int idPaciente) {
18         this.idPaciente = idPaciente;
19     }
20
21     public String getNombre() {
22         return this.nombre;
23     }
24
25     /**
26      *
27      * @param nombre
28      */
29     public void setNombre(String nombre) {
30         this.nombre = nombre;
31     }
32

```

6. Reflexiones y conclusiones

6.1 Ventajas e inconvenientes de la herramienta.

Considero que el uso de **Visual Paradigm** ha sido determinante para la calidad de mi proyecto. Como principal **ventaja**, destaco la capacidad de mantener una coherencia total: si modifico un método en el Diagrama de Clases, el cambio se propaga automáticamente a los diagramas de Comportamiento. Además, la generación automática de código Java me ha permitido ahorrar tiempo y evitar errores manuales de sintaxis.

Como **inconveniente**, he encontrado que la curva de aprendizaje inicial es pronunciada. Me resultó complejo, por ejemplo, configurar correctamente las propiedades de los mensajes en los diagramas de secuencia o gestionar las guardas en el diagrama de actividad para que el flujo fuera lógico.

6.2 Valoración del trabajo en equipo.

Mi experiencia grupal en esta actividad ha sido un proceso de aprendizaje crítico. La principal ventaja teórica de trabajar en equipo es la suma de perspectivas; sin embargo, en esta ocasión, el inconveniente principal fue la falta de consenso sobre la importancia de la coherencia técnica.

Al trabajar en grupo, a medida que avanzaba el proyecto, me di cuenta de que el modelado es un sistema integrado. Entendí que no se pueden adaptar las clases, métodos o atributos de forma arbitraria para que "encajen" en un diagrama de secuencia o de estados, sino que debe ser exactamente al revés: los diagramas de comportamiento deben derivar y respetar estrictamente la estructura definida en el modelo de clases.

Si estos diagramas no se siguen al pie de la letra, el modelado pierde su significado, su razón de ser y su utilidad para generar código real. Al ver que mis propuestas para mantener esta integridad eran desestimadas y que el trabajo perdía coherencia, decidí realizar este proyecto de forma individual. Aunque esto ha incrementado significativamente mi carga de trabajo, me ha dado la seguridad de entregar un sistema técnicamente sólido y alineado con los estándares profesionales que busco alcanzar.

Nota: Además de lo entregado en la carpeta comprimida, el trabajo se ha alojado en [github](#).